

INTER IIT TECH MEET 15.0 *PREPATHON PS*

AERIAL ROBOTICS

DCs: Divy Vikram Singh (8604796955), Krishnil Shindore (9922372478)

Vision-Based Autonomous Navigation for GPS-Denied UAV Operation

1. INTRODUCTION & MOTIVATION

Modern UAV applications increasingly demand operation in GPS-denied environments such as indoor facilities, urban canyons, underground structures, and areas with signal jamming. Traditional GPS-based navigation fails in these scenarios, creating critical operational gaps for surveillance, search-and-rescue, and mapping missions. Vision-based navigation represents the next frontier in autonomous UAV technology, enabling robust operation using onboard cameras and computer vision algorithms. However, current implementations face significant challenges including scale ambiguity, feature tracking failures, and integration with flight control systems. PX4 is one of the widely-used open-source flight controller stacks that provides the estimator infrastructure for this, though the vision pipeline and its integration are left entirely to the developer.

Recent developments in edge computing, AI-enhanced perception, and real-time processing have created new opportunities for reliable vision-only navigation. The ability to maintain stable flight, handle sensor failures gracefully, and provide accurate positioning without external infrastructure is crucial for next-generation autonomous systems.

As UAV operations expand into complex environments where GPS is unavailable or unreliable, developing robust vision-based navigation becomes essential for mission success and operational safety.

Separately, the simple and minimal design of quadrotors makes them vulnerable to the failure of a single motor. Real motor failures are rarely a clean, instantaneous stop – a worn bearing or a damaged propeller more commonly produces a progressive loss of thrust, and a vehicle can become meaningfully underactuated well before any rotor reaches zero output. Characterizing this behavior correctly, through the flight stack's own control-allocation representation rather than an ad-hoc override, is foundational to any future recovery work.

2. PROBLEM STATEMENT

Design and implement a complete vision-based autonomous navigation system capable of GPS-denied flight operations with robust failsafe handling and recovery mechanisms. Additionally, implement a runtime mechanism to modify a single rotor's effectiveness (a scalar applied to the selected rotor's complete effectiveness representation) in the PX4 flight stack, and characterize the resulting flight behavior across a range of degradation levels.

2.1 Part 1: Vision-Based Navigation System in SITL Simulation (45% of total points)

Vision Health Publisher Development

Implement MAVLink-based health monitoring that enables normal arming without force commands.

Robust State Estimation

Develop position and velocity estimation algorithms that handle lighting variations, motion blur, and feature tracking failures.

EKF Integration

Integrate vision estimates with PX4's EKF2 estimator to maintain variance thresholds below failsafe limits.

2.2 Part 2: PX4 Motor Effectiveness Characterization (55% of total points)

Autonomous Takeoff

Achieve stable autonomous takeoff and hover at 20 m altitude using the PX4 flight stack. Once stable, trigger the flight stack's own existing failure-handling capability once, to observe its behaviour as a reference point for later comparison.

Runtime Effectiveness Modification

Implement a mechanism that changes a selected rotor's effectiveness while the vehicle is flying, across a range of levels, without restarting the simulation.

Flight Demonstration

Characterize the vehicle's response across multiple effectiveness levels, from fully functional down to complete loss, without implementing failure detection or recovery algorithms.

PX4 Integration

Demonstrate understanding of PX4's control allocation system and actuator output pipeline, and of how it differs from the flight stack's own built-in failure handling.

Note

The requirements below get progressively harder within each part — later phases assume more from you than earlier ones. It's not compulsory to reach every level of every phase. A simpler or partial implementation, submitted with a clear and honest account of what you attempted and where it fell short, will be evaluated too. Submit what you have rather than nothing.

3. TECHNICAL REQUIREMENTS

3.1 Part 1: Vision-Based Navigation System

Phase 1: Vision Health Integration (20% of points)

- Publish external vision estimates (via `VISION_POSITION_ESTIMATE` or `ODOMETRY`, mapped to PX4's `vehicle_visual_odometry` topic) including velocity information.
- Achieve normal arming without "force" commands by providing stable health metrics.
- Demonstrate stable EKF2 integration with external vision fusion enabled.
- Provide real-time confidence metrics and quality assessment based on feature count and tracking stability.
- Configure proper parameter files with GPS disabled and appropriate EKF2 external vision fusion settings (`EKF2_EV_CTRL`, `EKF2_HGT_REF`).
- Maintain consistent 20-30 Hz message frequency with proper FRD/NED frame coordinate conversion.

Phase 2: Autonomous Position Hold (25% of points)

- Maintain hover within 1.5 m radius for 90 s using vision-only navigation in simulation.
- Handle EKF2 failsafe triggers gracefully, with automatic recovery once valid vision data resumes, when the estimator flags position or velocity fusion as degraded or invalid.
- Log synchronized vision/control data for performance analysis, including estimator innovation and health monitoring.
- Implement scale-consistent velocity estimation using known altitude (10 m takeoff height).
- Achieve stable estimator characteristics, keeping position and velocity innovations below failsafe thresholds throughout.

3.2 Part 2: PX4 Motor Effectiveness Characterization

Note

The PS doesn't ask for any stabilization or recovery. It is completely normal for the UAV to tumble and fall uncontrollably upon under-actuation.

Phase 1: PX4 Setup and Takeoff (10% of points)

- Configure PX4 firmware with standard quadrotor mixer configuration using Gazebo SITL simulation.
- Takeoff to 20 m altitude with all four motors operational.

- Once stable, trigger the flight stack's own existing failure-handling capability once, to observe its behavior as a reference for Phase 2.

Phase 2: Runtime Effectiveness Modification and Characterization (45% of points)

- Implement a runtime mechanism, of your own design, that changes a selected rotor's effectiveness within the control allocation model while the vehicle is airborne, without restarting the simulation.
- The mechanism must support at least the following effectiveness levels for the selected rotor: 100%, 75%, 50%, 25%, and 0%. It must genuinely update the allocator's effectiveness representation at the proper layer.
- Overriding final motor commands, actuator outputs, or the simulated motor model directly does not satisfy this requirement.
- Log the effectiveness values live as they change, so each transition is directly observable in your demonstration.
- Characterize the vehicle's flight response at each effectiveness level at 20 m altitude, without implementing any failure detection or recovery logic.
- Compare your complete-loss (0%) result against the reference behavior observed in Phase 1, and explain any difference in your report. Reproducing the reference's exact behavior is not required; a well-reasoned explanation of the comparison carries more weight than simply reaching all five levels.

4. INTEGRATION REQUIREMENTS

- Both parts must use the same simulation environment (Gazebo).
- Code quality standards including proper documentation, modular design, and parameter configuration.
- Real-time performance constraints: vision pipeline < 60 ms latency, control loops maintain standard rates.

5. SUBMISSION FORMAT

- A zip file containing 2 folders named PART_1 and PART_2.
- Each folder should contain:
 - The files you added or modified
 - A README.md written to the standard of a public GitHub repository: clear structure, setup/run instructions, and your approach and reasoning explained
 - Relevant logs and flowcharts (if needed)
 - Video demonstration

6. RESOURCES

- Basics of Quadcopter Controls by MATLAB: <https://in.mathworks.com/videos/series/drone-simulation-and-control.html>
- PX4 Dev Environment Setup in Linux: https://docs.px4.io/main/en/dev_setup/dev_env_linux_ubuntu.html
- PX4 Control Architecture: https://docs.px4.io/main/en/flight_stack/controller_diagrams
- PX4 Control Allocation: https://docs.px4.io/main/en/concept/control_allocation.html
- PX4 External Vision/Position Estimation: https://docs.px4.io/main/en/ros/external_position_estimation.html
- PX4 EKF2 Tuning: https://docs.px4.io/main/en/advanced_config/tuning_the_ecl_ekf.html
- PX4 Failure Injection (for study/reference only): https://docs.px4.io/main/en/debug/failure_injection.html